

GNN-EGG: Graph Neural Network Explanations *via* Graph Generation

Art Tay

Abstract

Keywords: Graph Neural Networks, Interpretable AI

Introduction

Related Work

Methodology

Notation

Let G be a graph with X , E , and A being the node feature matrix, edge feature matrix, and adjacency matrix respectively. Let $X = [X_c, X_d]$ and $E = [E_c, E_d]$ where c and d denote continuous and one-hot discrete features.

Generation

X_c and E_c can both be sampled from any function with the form:

$$f(z ; \Omega) : \mathbb{R}^Z \rightarrow \mathbb{R}^{dim}$$

Where z is a random, Ω is a set of potentially trainable parameters, and dim denotes the dimensionality of the matrix we want to generate. z could be sampled as pure random noise, but z could also be a function of the data. In most instances, we generate continuous feature matrices using a random vector from a multivariate normal pass through a standard dense layer with no activation.

X_d and E_d can both be sampled from the *concrete distribution* (Maddison, Mnih, and Teh 2017). Using the reparameterization trick and continuous relaxation, we can sample one-hot categorical vectors based on two parameters θ_{Cat} , a trainable parameter vector of length equal to the number of categories, and τ , a hyperparameter that controls the degree of relaxation (smaller value approximate the discrete sampling better, but can result in numerical issues).

$$\text{Softmax} \left(\frac{\theta - \log(-\log \epsilon)}{\tau} \right), \quad \epsilon \sim U[0, 1]$$

Like z , θ_{Cat} could be a function of the data and other trainable parameters. One concrete distribution would be required per discrete feature.

Similarly, the adjacency matrix can be sampled from the *binary concrete distribution*:

$$\text{sigmoid} \left(\frac{\theta + \log \epsilon - \log(1 - \log \epsilon)}{\tau} \right)$$

Identical to Wang and Shen (2023), set θ to be the same dimensionality as A , where each entry in θ is the probability that an edge exists between nodes i and j .

Pseudo-code for a generic *egg* generator is provided below.

```

generator(z_1, z_2, theta_1, theta_3, theta_3, tau_1, tau_2, tau_3){
    X_c = f(z_1; Omega_1)

    E_c = f(z_2; Omega_2)

    for each discrete node feature i do:
        X_di ~ concrete_1(theta_1i, tau_1)
        X_d = [X_d, X_di]

    for each discrete edge feature i do:
        E_di ~ concrete_1(theta_2i, tau_2)
        E_d = [E_d, E_di]

    A ~ binary_concrete(theta_3, tau_3)

    return X_c, X_d, E_c, E_d, A
}

```

Loss Function

GNN-EGG trains based on a three part loss function.

GNN Prediction & Embedding Loss

Let `explaineer()` be the GNN model we want to explain. Let $g()$ represent the embedding layers and $h()$ represent the dense predictions layers. $g()$ is assumed to return a vectorization of a graph, and $h()$ is assumed to return a class probability vector.

$$\mathcal{L}_p = \text{distance}(g(G), \psi) + \text{Cross-Entropy}(h(g(G)), t)$$

t denotes the desired prediction, and ψ can either be the average embedding for the target class, like Wang and Shen (2023) or a particular observation.

Most GNN `explaineer` model will require further discretization of the `generator`'s output. The typically transformation are $A' = I(A \geq 0.5)$, $X_d = \text{argmax}(X_d)$, $E_c = \text{argmax}(E_c)$; however, each one will prevent gradient backpropagation with respect to their corresponding parameters. This issue can be mitigated in two ways. First, the remaining two loss function terms do not involve the `explaineer` directly and are designed to use the `generator` output without transformation. Second, a REINFORCE (Williams 1992) term can be added based on the likelihood of the generating distributions. For example if the A' transformation is required, for following term can be added to ensure training of the edge probabilities.

$$\mathcal{L}_R = -\frac{l(\theta_A | G)}{\mathcal{L}}$$

Here $\mathcal{L}$ denotes the sum of all other loss terms and l denotes the log-likelihood. In short, this term moves θ_A to move the likelihood of G proportional to the inverse of the loss ie if $\mathcal{L}(G)$ is small, θ_A will be moved to increase the likelihood of G and vice-a-versa.

Regularization & Sparsity

Regularization the probability of edges existing will increase sparsity in terms of edges, and nodes if we remove any that become *isolated*. Identical to the regularization done in Wang and Shen (2023), we add the

following term.

$$\mathcal{L}_s = \|\theta_A\|^2 + \text{softplus}(\|\text{sigmoid}\|_1 - B)^2$$

B here is usually called the budget. It represents a ceiling on the number of edges that we desire on generated examples.

Structural Consistency Loss

Structurally consistency is the hardest to train, but arguable the most important property for an example to have. Most GNN use a message passing paradigm followed by pooling and dense prediction layers. The idea is to use the graph’s structure to engineer a better node features to draw predictions. This principle exposes a flaw in the generation scheme. If the generator is allowed to train the node features directly, it becomes likely that the generator will ignore the graph structure and leap to training itself to generate the final features needed for prediction.

[A good example of this happening is that a model training only the node features, with all else random, was able to produce and incredible tight prediction distribution. This could also be done in spite of extreme regularization.]

Using embeddings in addition to the prediction can mitigate this problem; however, it is possible that embeddings could also be subject to this problem. For large graphs, there is a considerable amount of information that can be unrepresented in the latent vector. Two graphs could be very different on the graph level, but have very similar embeddings. Furthermore, measurements of similarity of the latent space aren’t readily interpretable by humans making is harder to evaluate and compare the realism of the generated data.

Results

References

- Maddison, Chris J., Andriy Mnih, and Yee Whye Teh. 2017. “The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables,” no. arXiv:1611.00712 (March). <https://doi.org/10.48550/arXiv.1611.00712>.
- Wang, Xiaoqi, and Han-Wei Shen. 2023. “GNNInterpreter: A Probabilistic Generative Model-Level Explanation for Graph Neural Networks,” no. arXiv:2209.07924 (February). <https://doi.org/10.48550/arXiv.2209.07924>.
- Williams, Ronald J. 1992. “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.”